

■ Copa do Mundo FIFA 2026

Documentação Técnica do Projeto de Análise de Dados

Como extraímos e analisamos dados reais da API da FIFA

Data	26 de junho de 2026
Autor	Lucas Martins
Linguagem	Python 3.12
Fonte	API pública oficial FIFA (api.fifa.com/api/v3)
Jogos base	1.068 partidas (Copa do Mundo 1930–2026)

Este documento explica, de forma acessível a iniciantes em programação, como foram construídos os scripts Python que coletam, processam e analisam dados reais da Copa do Mundo FIFA 2026 diretamente da API oficial da FIFA.

1. O que é uma API e como a da FIFA funciona

Uma **API** (Interface de Programação de Aplicações) é como um cardápio de restaurante: você faz um pedido específico e recebe exatamente o que pediu, sem precisar entrar na cozinha. No nosso caso, fazemos pedidos ao servidor da FIFA e recebemos dados de jogos, times e estatísticas.

A FIFA possui uma API pública que alimenta o próprio site *inside.fifa.com*. Ela não exige cadastro nem chave de acesso — basta simular que somos um navegador enviando os cabeçalhos (headers) corretos na requisição:

```
import requests

sessao = requests.Session()

sessao.headers.update({
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64)...",
    "Origin": "https://inside.fifa.com",
    "Referer": "https://inside.fifa.com/",
    "Accept": "application/json",
})
```

O que são esses cabeçalhos? O *User-Agent* diz ao servidor qual navegador estamos usando (fingimos ser o Chrome). O *Origin* e o *Referer* dizem de qual site estamos vindo. Com isso, o servidor da FIFA nos responde normalmente, pois pensa que somos o próprio site deles.

Endpoints utilizados

Um **endpoint** é o endereço específico de cada tipo de dado na API. É como cada seção do cardápio. Usamos cinco:

Endpoint	O que retorna
/seasons?idCompetition=17	Lista das 23 edições da Copa do Mundo
/calendar/matches?idSeason={id}	Todos os jogos de uma edição
/timelines/17/{season}/{stage}/{match}	Eventos minuto a minuto de um jogo
/topscorers?idSeason=285023	Artilheiros com gols, chutes e assist.
/topcards?idSeason=285023	Cartões e faltas por jogador

■ ■ *idCompetition=17 = Copa do Mundo masculina | idSeason=285023 = edição 2026*

2. Extração de Jogos (fifa_extrair_jogos.py)

Este script é o ponto de partida do projeto. Ele baixa todos os jogos de todas as edições da Copa do Mundo, desde 1930 até 2026, e salva os dados em arquivos para uso posterior.

2.1 Como o script funciona (passo a passo)

Passo	O que acontece
1	Consulta /seasons e descobre as 23 edições (1930–2026)
2	Para cada edição, consulta /calendar/matches e baixa os jogos
3	Salva tudo em fifa_jogos_completo.json (1.068 jogos no total)
4	Gera fifa_jogos.csv com datas legíveis e coluna de vencedor
5	Gera fifa_previsao_campeao_2026.csv com ranking por pontos

2.2 Por que os nomes dos times vêm em lista?

A FIFA serve o conteúdo em vários idiomas ao mesmo tempo. Por isso, o nome de um time não vem como texto simples, mas como uma **lista de traduções**:

```
"TeamName": [  
  { "Locale": "pt-BR", "Description": "França" },  
  { "Locale": "en-GB", "Description": "France" },  
  { "Locale": "es-ES", "Description": "Francia" }  
]
```

Criamos uma função que percorre essa lista e devolve o texto no idioma desejado (português), caindo para o primeiro disponível se não encontrar:

```
def texto_localizado(lista, idioma="pt"):  
    for item in lista:  
        if item["Locale"].lower().startswith(idioma):  
            return item["Description"]  
    return lista[0]["Description"] # fallback
```

2.3 Unificação de times históricos

Ao longo da história, alguns países mudaram de nome. Para não contar 'Alemanha' e 'Alemanha Ocidental' como times separados, criamos uma função de padronização:

```
def padroniza_nome(nome):  
    n = nome.lower().strip()  
    # Alemanha + Alemanha Ocidental (RFA) viram uma só  
    if "aleman" in n or "germany" in n or "rfa" in n:  
        return "Alemanha"  
    mapa = {  
        "uniao sovietica": "Russia",
```

```
"iugoslavia":      "Servia",  
}  
return mapa.get(n, nome) # se não estiver no mapa, devolve original
```

3. Modelo de Palpites com Distribuição de Poisson (fifa_jogos_hoje.py)

Para prever o resultado de um jogo, usamos um modelo matemático chamado **distribuição de Poisson**. Pode parecer complicado, mas a ideia central é simples: gols são eventos raros que acontecem de forma independente ao longo do jogo — exatamente o que a distribuição de Poisson modela.

3.1 Analogia para iniciantes

Pense assim: se um time marca em média 2 gols por jogo, qual a chance de marcar exatamente 3? A fórmula de Poisson responde isso. Fazemos isso para todos os placares possíveis (0×0, 0×1, 1×0, 2×0...) e somamos as probabilidades.

3.2 Calculando a força de cada seleção

Antes de prever, medimos o desempenho de cada time nos jogos já disputados. Calculamos dois índices: **força de ataque** (quão bem o time marca) e **força de defesa** (quão bem o time evita gols).

```
K = 4.0 # regularização – suaviza times com poucos jogos

ataque[time] = ((gols_marcados + K × média_geral)
                / (jogos_disputados + K)) / média_geral

defesa[time] = ((gols_sofridos + K × média_geral)
               / (jogos_disputados + K)) / média_geral
```

O valor **K=4.0** é a regularização: evita que um time que só jogou 1 partida e goleou por 5×0 apareça como o mais forte do torneio. Ele puxa o índice em direção à média geral quando há poucos dados.

3.3 Gols esperados no confronto

```
# Quantos gols cada time tende a marcar neste jogo específico

λ_mandante = média × ataque[mandante] × defesa[visitante]
λ_visitante = média × ataque[visitante] × defesa[mandante]
```

O símbolo λ (**lambda**) representa o número esperado de gols. Se o mandante é forte no ataque e o visitante é fraco na defesa, λ sobe. Se o visitante tem uma defesa sólida, λ cai.

3.4 Calculando as probabilidades

```
# Para cada placar possível (0x0 até 10x10):
for gols_casa in range(11):
    for gols_fora in range(11):
        # Probabilidade desse placar exato acontecer
        p = poisson(gols_casa, λ_mandante) × poisson(gols_fora, λ_visitante)

        if gols_casa > gols_fora: p_vitória_mandante += p
        if gols_casa == gols_fora: p_empate += p
        if gols_casa < gols_fora: p_vitória_visitante += p
```

No final, somamos todas as probabilidades de cada resultado. O palpite é o resultado com maior probabilidade. O placar previsto é o placar específico mais provável dentro do resultado escolhido.

4. Simulação Monte Carlo — Previsão do Campeão

Para prever o campeão, não basta calcular as probabilidades de um jogo. Precisamos simular o torneio inteiro, considerando todas as combinações possíveis de resultados. Fazemos isso 50.000 vezes.

4.1 O que é Monte Carlo?

Monte Carlo é uma técnica que usa **aleatoriedade controlada** para estimar probabilidades complexas. O nome vem do famoso cassino de Mônaco — assim como lançar um dado muitas vezes nos dá a frequência de cada face, simular o torneio muitas vezes nos dá a frequência de cada campeão.

Exemplo simples: Se quisermos saber a probabilidade de tirar duas caras seguidas ao jogar uma moeda, podemos calcular matematicamente (25%) ou simplesmente jogar a moeda 10.000 vezes e contar. Monte Carlo é a segunda abordagem aplicada ao futebol.

4.2 Fluxo de cada simulação

Etapa	O que acontece
1 — Fase de grupos	Simula os jogos pendentes. Para cada jogo, amostra os gols de uma distribuição de Poisson aleatoriamente.
2 — Classificação	Ordena os times por pontos → saldo → gols. Top 2 de cada grupo + 8 melhores 3os = 32 classificados.
3 — Mata-mata	Simula 32 → 16 → 8 → 4 → 2 → 1. Empates no tempo normal vão a pênaltis (probabilidade por força ofensiva).
4 — Registra campeão	O time que sobrou é marcado como campeão desta simulação.
5 — Repete 50.000×	Ao final, a % de título = (vitórias do time) ÷ 50.000 × 100.

4.3 Como sortear gols aleatoriamente (Poisson)

```
def amostrar_poisson(lam):  
    # Método de Knuth – simula quantos gols acontecem  
    # dado que a taxa esperada é `lam`  
    L = math.exp(-lam) # limiar de parada  
    k, p = 0, 1.0  
    while p > L:  
        p *= random.random() # multiplica probabilidades  
        k += 1  
    return k - 1 # número de gols simulado
```

Este algoritmo é matematicamente equivalente a usar a fórmula de Poisson, mas muito mais eficiente para gerar números aleatórios em larga escala.

4.4 Simulação de pênaltis

```
# Times mais ofensivos têm leve vantagem nos pênaltis  
prob_time_a = ataque[time_a] / (ataque[time_a] + ataque[time_b])  
vencedor = time_a if random.random() < prob_time_a else time_b
```

5. Extração de Eventos por Jogo (/timelines)

O endpoint mais rico da API é o **/timelines**. Ele retorna todos os eventos de um jogo — gols, faltas, substituições, cartões — com o minuto exato e o jogador envolvido.

5.1 Como fazer a requisição

```
# Precisamos de 4 IDs para montar a URL:
# idCompetition, idSeason, idStage, idMatch

url = f"https://api.fifa.com/api/v3/timelines/17/285023/{idStage}/{idMatch}"

resposta = sessao.get(url, params={"language": "pt"})
eventos = resposta.json().get("Event", [])

# eventos é uma lista com ~80 itens por jogo
```

5.2 Estrutura de um evento

Cada evento no JSON tem esta estrutura. O campo **Type** (número) identifica o que aconteceu, e o **EventDescription** traz o texto em português com o nome do jogador:

```
{
  "Type": 0,          ← 0 = gol
  "MatchMinute": "9'",
  "IdPlayer": "429157",
  "IdTeam": "43911",
  "EventDescription": [
    {
      "Locale": "pt-BR",
      "Description": "Julian QUINONES (México) marca o gol!!"
    }
  ]
}
```

5.3 Todos os tipos de eventos encontrados

Varremos todos os 64 jogos disputados da Copa 2026 para mapear os tipos de eventos existentes:

Tipo	Evento	Total nos 64 jogos	Tem jogador?
0	Gol	169	Sim
1	Assistência	136	Sim
2	Cartão amarelo	150	Sim
3	Cartão vermelho	9	Sim
5	Substituição	592	Sim
12	Chute a gol	1.531	Sim
15	Impedimento	209	Sim

Tipo	Evento	Total nos 64 jogos	Tem jogador?
16	Escanteio	555	Sim
18	Falta	1.259	Sim
34	Gol contra	12	Sim
41	Gol de pênalti	6	Sim
57	Gol evitado	312	Sim
71	VAR	18	Não
79/80	Cara ou coroa	64	Não

5.4 Extrair o nome do jogador com Regex

O nome do jogador vem embutido no texto do EventDescription, no formato '*NOME (Seleção) fez algo*'. Usamos **expressão regular (regex)** para extrair o nome e o time:

```
import re

def nome_jogador(evento):
    desc = texto_localizado(evento.get("EventDescription"))
    # Padrão: qualquer texto + (qualquer texto dentro dos parênteses)
    m = re.match(r'^([^\s]+)\s+([^\s]+)', desc.strip())
    if m:
        nome = m.group(1).strip().title() # "Julian Quinones"
        time = m.group(2).strip()         # "México"
        return nome, time
    return desc[:40], "" # fallback se o padrão não bater
```

Como funciona o regex: `^([^\s]+)` captura tudo antes do primeiro parêntese (o nome). `\s+([^\s]+)` captura o conteúdo entre parênteses (o time). O método `.title()` coloca a primeira letra de cada palavra em maiúscula.

6. Estatísticas Oficiais da FIFA (/topscorers e /topcards)

Além dos eventos por jogo, a FIFA disponibiliza estatísticas agregadas por jogador em dois endpoints oficiais — dados que a própria FIFA usa em seu site.

6.1 /topscorers — Artilheiros e estatísticas ofensivas

```
resposta = sessao.get(
    "https://api.fifa.com/api/v3/topscorers",
    params={
        "idCompetition": "17",
        "idSeason":      "285023",
        "language":      "pt",
        "count":          200
    }
)
jogadores = resposta.json().get("Results", [])
```

Campos retornados por jogador:

Campo	Significado
Goals	Total de gols marcados
Assists	Total de assistências
Shots	Total de chutes (no alvo + fora)
AttemptsOnTarget	Chutes no alvo (que o goleiro defendeu ou entraram)
Matches	Número de jogos disputados
MinutesPlayed	Total de minutos em campo
PenaltiesScored	Pênaltis convertidos

6.2 /topcards — Disciplina e faltas

Campo	Significado
YellowCards	Cartões amarelos recebidos
RedCards	Cartões vermelhos diretos
DoubleYellowCards	Expulsões por segundo amarelo
FoulsCommitted	Faltas cometidas
FoulsSuffered	Faltas sofridas
PenaltyFouls	Faltas que geraram pênalti

■ ■ Endpoints de 'Man of the Match' (/motm, /awards, /ratings) foram todos testados e retornam erro 404 — a FIFA não expõe esse dado na API pública.

7. Sistema de Pontuação e Métricas

Como a FIFA não disponibiliza um 'destaque por jogo' oficial, criamos uma **pontuação ponderada** que combina os eventos de cada jogador em uma nota única. Quanto maior a pontuação, mais impacto o jogador teve na partida.

7.1 Tabela de pesos

Evento	Pontos	Por quê?
Gol / Gol de pênalti	+4	Ação mais decisiva do futebol
Assistência	+2	Passe direto que originou o gol
Chute no alvo	+1	Tentativa real de marcar
Gol evitado (defesa)	+1	Oportunidade criada que foi parada
Falta cometida	-0,5	Interrupção de jogo e risco de cartão
Cartão amarelo	-1	Advertência oficial
Cartão vermelho	-3	Expulsão — prejudica gravemente o time

7.2 Participações em Gol (Goal Contributions)

Métrica padrão internacional usada por sites como SofaScore, WhoScored e ESPN:

```
Participações em Gol = Gols + Assistências
```

Exemplo: Mbappé tem 4 gols + 2 assistências = **6 Participações em Gol**, o maior número da Copa 2026 até agora.

7.3 Chances Criadas

Soma de todas as ações ofensivas que geraram uma oportunidade real:

```
Chances Criadas = Gols + Assistências + Chutes + Gols Evitados
```

7.4 Cruzamento dos endpoints oficiais

```
# Buscamos os dois endpoints
scorers = sessao.get("/topscorers", params=...).json()["Results"]
cartoes = sessao.get("/topcards", params=...).json()["Results"]

# Criamos um dicionário para busca rápida por ID do jogador
cartoes_map = {p["IdPlayer"]: p for p in cartoes}

# Para cada jogador em scorers, cruzamos com os dados de cartões
for jogador in scorers:
    pid = jogador["IdPlayer"]
    dados_c = cartoes_map.get(pid, {}) # {} se não tiver cartão

    gols = jogador.get("Goals", 0) or 0
    assist = jogador.get("Assists", 0) or 0
```

```
amarelo = dados_c.get("YellowCards", 0) or 0
```

```
pontuacao = gols*4 + assist*2 + amarelo*(-1) + ...
```

8. Top 20 — Ranking Final com Dados Oficiais FIFA

Resultado final do ranking combinando /topscorers e /topcards, ordenado por Pontuação (dados oficiais da FIFA, Copa 2026, fase de grupos):

#	Jogador	País	Gols	Assist.	Part. em Gol	Chutes	No Alvo	Pont.
1	Kylian Mbappé	FRA	4	2	6	16	9	29,0
2	Lionel Messi	ARG	5	0	5	13	8	28,0
3	Vinicius Junior	BRA	4	1	5	12	8	26,0
4	Ousmane Dembélé	FRA	4	1	5	8	5	23,0
5	Erling Haaland	NOR	4	0	4	9	6	22,0
6	Deniz Undav	GER	3	2	5	5	4	20,0
7	Ismaila Sarr	SEN	3	1	4	12	5	19,0
8	Jonathan David	CAN	3	0	3	13	7	19,0
9	Johan Manzambi	SUI	3	1	4	6	3	17,0
10	Alexander Isak	SWE	1	3	4	6	5	15,0
11	Brian Brobbey	NED	3	0	3	4	3	15,0
12	Matheus Cunha	BRA	3	0	3	6	3	15,0
13	Ismael Saibari	MAR	3	0	3	10	3	15,0
14	Cody Gakpo	NED	2	1	3	9	5	15,0
15	Mikel Oyarzabal	ESP	2	1	3	10	4	14,0
16	Ruben Vargas	SUI	2	1	3	6	4	14,0
17	Maxi Araujo	URU	2	1	3	4	3	13,0
18	Ayase Ueda	JPN	2	1	3	7	3	13,0
19	Cristiano Ronaldo	POR	2	0	2	10	5	13,0
20	Harry Kane	ENG	2	0	2	10	4	12,0

9. Fluxo Completo do Projeto

Visão geral de como todos os scripts e dados se conectam:

Etapa	Script / Endpoint	Entrada	Saída
1 — Baixar jogos	fifa_extrair_jogos.py /seasons + /calendar/matches	—	fifa_jogos_completo.json fifa_jogos.csv
2 — Palpites do dia	fifa_jogos_hoje.py (usa Poisson)	fifa_jogos_completo.json	fifa_palpite_hoje.csv
3 — Prever campeão	Simulação Monte Carlo 50.000 iterações	fifa_jogos_completo.json	Ranking com % por time
4 — Eventos por jogo	/timelines (64 jogos)	IDs dos jogos	fifa_artilharia.csv fifa_faltas.csv fifa_chutes.csv fifa_destaque_jogo.csv
5 — Stats oficiais	/topscorers + /topcards	idSeason=285023	Ranking Top 20 com pontuação

10. Glossário para Iniciantes

Termo	O que significa
API	Interface para buscar dados de um servidor via internet
Endpoint	URL específica de um tipo de dado na API
JSON	Formato de texto estruturado para troca de dados (como uma planilha, mas em texto)
CSV	Arquivo de texto com dados separados por vírgula (abre no Excel)
Poisson	Distribuição matemática que modela eventos raros e independentes (como gols)
Monte Carlo	Técnica de simulação que usa aleatoriedade para estimar probabilidades
Lambda (λ)	Símbolo matemático para a taxa média esperada de um evento (ex: gols por jogo)
Regex	Expressão regular — padrão de texto para extrair informações de strings
Header	Cabeçalho enviado junto com a requisição HTTP (identifica quem está pedindo)
regularização	Técnica para evitar distorções quando há poucos dados disponíveis (o fator K=4)
Goal Contributions	Participações em gol = Gols + Assistências (métrica internacional padrão)

11. Mergulho no Código — Parte 1: Fazendo Requisições

Vamos entender linha por linha como o Python se comunica com a API da FIFA. A biblioteca usada é o **requests**, a mais popular para fazer requisições HTTP em Python.

11.1 O que é uma requisição HTTP?

Quando você abre um site no navegador, ele faz uma **requisição HTTP GET** ao servidor — basicamente pergunta: 'me dá esse conteúdo?'. Nosso código faz exatamente isso, mas em vez de receber HTML (página), recebe JSON (dados puros).

```
import requests # biblioteca para fazer requisições HTTP

# Session() mantém as configurações para todas as requisições
# (como fazer login uma vez e ficar logado)
sessao = requests.Session()

# headers = informações que enviamos junto com a requisição
# O servidor usa isso para decidir o que nos responder
sessao.headers.update({
    "User-Agent": "Mozilla/5.0...", # simula um navegador Chrome
    "Origin": "https://inside.fifa.com", # finge que veio desse site
    "Referer": "https://inside.fifa.com/",
    "Accept": "application/json", # diz que queremos JSON de volta
})
```

11.2 Fazendo a requisição e lendo a resposta

```
url = "https://api.fifa.com/api/v3/calendar/matches"

# params = parâmetros que vão na URL (depois do ?)
# Resultado: .../calendar/matches?idCompetition=17&idSeason;=285023
params = {
    "idCompetition": "17",
    "idSeason": "285023",
    "count": 1000,
    "language": "pt",
}

resposta = sessao.get(url, params=params, timeout=30)
# timeout=30 → se demorar mais de 30 segundos, desiste

# Verificando se deu certo (status 200 = OK, 404 = não encontrado)
print(resposta.status_code) # → 200

# Convertendo o texto JSON em dicionário Python
dados = resposta.json()
jogos = dados.get("Results", []) # lista de jogos
print(f"Jogos encontrados: {len(jogos)}")
```

11.3 Retry automático — tentativas em caso de falha

A API pode falhar ocasionalmente. Criamos uma função que tenta até 4 vezes antes de desistir, esperando um tempo maior a cada tentativa:

```
def get(url, params=None, tentativas=4):
    for n in range(1, tentativas + 1):
        resposta = sessao.get(url, params=params, timeout=30)

        # .ok = True se status HTTP for 200-299
        if resposta.ok and resposta.text.strip():
            try:
                return resposta.json() # sucesso!
            except ValueError:
                pass # resposta veio mas não era JSON válido

        # Espera mais a cada tentativa: 1.5s, 3s, 4.5s, 6s
        espera = 1.5 * n
        print(f"Tentativa {n} falhou. Aguardando {espera}s...")
        time.sleep(espera)

    raise RuntimeError("API não respondeu após 4 tentativas.")
```

12. Mergulho no Código — Parte 2: Salvando e Lendo Arquivos

12.1 Salvando em JSON

JSON é o formato bruto dos dados — guardamos tudo sem perder nenhum campo. Serve como backup completo e é a entrada dos outros scripts.

```
import json

# SALVAR: converte o dicionário Python em texto JSON e grava no arquivo
with open("fifa_jogos_completo.json", "w", encoding="utf-8") as arquivo:
    json.dump(dados, arquivo, ensure_ascii=False, indent=2)

# ensure_ascii=False → mantém acentos (ã, é, ç...)
# indent=2 → formata com indentação legível

# LER: lê o arquivo e converte de volta para dicionário Python
with open("fifa_jogos_completo.json", "r", encoding="utf-8") as arquivo:
    dados = json.load(arquivo)

print(type(dados))  # → <class 'list'>
print(len(dados))   # → 1068
```

12.2 Salvando em CSV (planilha)

CSV é uma planilha em formato de texto. Pode ser aberto no Excel, Google Sheets ou qualquer programa de análise de dados.

```
import csv

linhas = [
    {"time_casa": "Brasil", "gols_casa": 3, "time_fora": "Escócia", "gols_fora": 0},
    {"time_casa": "França", "gols_casa": 4, "time_fora": "Noruega", "gols_fora": 1},
]

# SALVAR CSV
with open("fifa_jogos.csv", "w", newline="", encoding="utf-8-sig") as arquivo:
    # utf-8-sig = UTF-8 com BOM (evita problema de acentos no Excel)
    escritor = csv.DictWriter(arquivo, fieldnames=list(linhas[0].keys()))
    escritor.writeheader()  # escreve a linha de cabeçalho
    escritor.writerows(linhas)  # escreve todas as linhas de dados

# LER CSV
with open("fifa_jogos.csv", "r", encoding="utf-8-sig") as arquivo:
    leitor = csv.DictReader(arquivo)
    for linha in leitor:
        print(linha["time_casa"], linha["gols_casa"])
```


13. Mergulho no Código — Parte 3: Processando os Dados

13.1 defaultdict — acumulador sem erro

Ao acumular estatísticas por jogador, um dicionário normal causaria erro ao tentar acessar uma chave que ainda não existe. O **defaultdict** cria o valor padrão automaticamente:

```
from collections import defaultdict

# Dicionário normal – ERRO se a chave não existir:
gols = {}
gols["Messi"] += 1    # KeyError: 'Messi'

# defaultdict – cria o valor padrão automaticamente:
gols = defaultdict(int)    # int() = 0
gols["Messi"] += 1        # funciona! gols["Messi"] = 1
gols["Haaland"] += 2      # funciona! gols["Haaland"] = 2

# Usamos lambda para valores mais complexos:
stats = defaultdict(lambda: {"gols": 0, "faltas": 0, "time": ""})
stats["Vinicius"]["gols"] += 1
stats["Vinicius"]["time"] = "Brasil"
print(stats["Vinicius"])  # → {"gols": 1, "faltas": 0, "time": "Brasil"}
```

13.2 Filtrando jogos já disputados

A API retorna tanto jogos passados quanto futuros. Precisamos filtrar apenas os já disputados, comparando a data do jogo com o momento atual:

```
from datetime import datetime, timezone

# Momento atual em UTC (padrão internacional de fuso horário)
agora = datetime.now(timezone.utc).replace(tzinfo=None)

def ja_disputado(jogo):
    iso = jogo.get("Date")          # ex: "2026-06-11T19:00:00Z"
    if not iso:
        return False
    # Converte string ISO para objeto datetime
    data_jogo = datetime.strptime(str(iso)[:19], "%Y-%m-%dT%H:%M:%S")
    return data_jogo < agora        # True se já passou

# Filtrando a lista de jogos:
disputados = [j for j in todos_jogos if ja_disputado(j)]
futuros    = [j for j in todos_jogos if not ja_disputado(j)]
print(f"Disputados: {len(disputados)} | Futuros: {len(futuros)}")
```

13.3 Convertendo fuso horário para Brasília

A FIFA entrega todas as datas em **UTC** (horário universal). Convertemos para o horário de Brasília (UTC-3) somando o offset:

```
from datetime import timedelta

FUSO_HORARIO = -3    # Brasília = UTC - 3 horas

def para_horario_local(iso_utc):

    data_utc = datetime.strptime(str(iso_utc)[:19], "%Y-%m-%dT%H:%M:%S")
    data_local = data_utc + timedelta(hours=FUSO_HORARIO)
    return data_local

# Exemplo:
# UTC:      2026-06-11T19:00:00Z    (19h em Londres)
# Brasília: 2026-06-11 16:00      (16h no Brasil)
```

14. Mergulho no Código — Parte 4: Lógica dos Eventos de Timeline

14.1 Baixando e processando 64 jogos em sequência

Para extrair as estatísticas de todos os jogos, percorremos a lista de partidas disputadas e fazemos uma requisição por jogo, acumulando os dados:

```
import time

# Acumuladores globais (somam dados de todos os jogos)
artilharia = defaultdict(lambda: {"gols": 0, "time": ""})
faltas_j = defaultdict(lambda: {"faltas": 0, "time": ""})

for jogo in jogos_disputados:
    # Extraindo os 4 IDs necessários para montar a URL
    idc = jogo.get("IdCompetition") # "17"
    ids = jogo.get("IdSeason")      # "285023"
    idst = jogo.get("IdStage")      # "289273"
    idm = jogo.get("IdMatch")       # "400021443"

    url = f"https://api.fifa.com/api/v3/timelines/{idc}/{ids}/{idst}/{idm}"
    resposta = sessao.get(url, params={"language": "pt"})
    eventos = resposta.json().get("Event", [])

    # Processa cada evento do jogo
    for evento in eventos:
        tipo = evento.get("Type")
        nome, equipe = extrair_nome(evento)

        if tipo == 0: # gol
            artilharia[nome]["gols"] += 1
            artilharia[nome]["time"] = equipe
        elif tipo == 18: # falta
            faltas_j[nome]["faltas"] += 1
            faltas_j[nome]["time"] = equipe

    time.sleep(0.25) # pausa para não sobrecarregar o servidor
```

14.2 Calculando o Score por jogo

Para cada jogo, calculamos o score de cada jogador separadamente. Ao final, o jogador com maior score é o destaque daquela partida:

```
# Para cada jogo, criamos um dicionário zerado por jogador
score_jogo = defaultdict(lambda: {
    "score": 0, "gols": 0, "assistencias": 0,
    "chutes": 0, "chances_criadas": 0, "time": ""
})

# Tabela de pontos por evento
```

```
PONTOS = {
    0: +4,    # gol
    41: +4,   # gol de pênalti
    1: +2,    # assistência
    12: +1,   # chute a gol
    57: +1,   # gol evitado
    18: -0.5, # falta cometida
    2: -1,    # cartão amarelo
    3: -3,    # cartão vermelho
}

for evento in eventos:
    tipo = evento.get("Type")
    nome, equipe = extrair_nome(evento)
    if not nome:
        continue

    pontos = PONTOS.get(tipo, 0)
    score_jogo[nome]["score"] += pontos
    if not score_jogo[nome]["time"]:
        score_jogo[nome]["time"] = equipe

# Quem teve o maior score nesse jogo?
destaque_nome = max(score_jogo, key=lambda n: score_jogo[n]["score"])
destaque_dados = score_jogo[destaque_nome]
print(f"Destaque: {destaque_nome} – Score: {destaque_dados['score']}")
```

15. Mergulho no Código — Parte 5: Modelo de Poisson Detalhado

15.1 A fórmula de Poisson explicada

A fórmula calcula: 'qual a probabilidade de ocorrerem exatamente k eventos, se a taxa média esperada é λ ?'

```
import math

def poisson_pmf(k, lam):
    # Fórmula:  $P(k) = e^{-\lambda} \times \lambda^k / k!$ 
    #
    #  $e^{-\lambda}$  → fator de decaimento (quanto maior  $\lambda$ , mais gols esperados)
    #  $\lambda^k$  →  $\lambda$  elevado a  $k$  (probabilidade de  $k$  eventos)
    #  $k!$  → fatorial de  $k$  (divide pelas formas de ordenar  $k$  eventos)
    return math.exp(-lam) * (lam ** k) / math.factorial(k)

# Exemplos práticos:
lam = 1.5 # time marca em média 1.5 gols por jogo

print(f"P(0 gols) = {poisson_pmf(0, lam):.1%}") # → 22.3%
print(f"P(1 gol) = {poisson_pmf(1, lam):.1%}") # → 33.5%
print(f"P(2 gols) = {poisson_pmf(2, lam):.1%}") # → 25.1%
print(f"P(3 gols) = {poisson_pmf(3, lam):.1%}") # → 12.6%
print(f"P(4 gols) = {poisson_pmf(4, lam):.1%}") # → 4.7%
```

15.2 Calculando probabilidades do confronto completo

Para um jogo entre dois times, calculamos a probabilidade de cada combinação de placar e somamos por resultado:

```
MAX_GOLS = 10 # placar máximo considerado (0 a 10 gols)

p_vitoria_casa = p_empate = p_vitoria_fora = 0.0

for gols_casa in range(MAX_GOLS + 1):
    # Probabilidade do time da casa marcar exatamente gols_casa
    p_casa = poisson_pmf(gols_casa, lambda_casa)

    for gols_fora in range(MAX_GOLS + 1):
        # Probabilidade do visitante marcar exatamente gols_fora
        p_fora = poisson_pmf(gols_fora, lambda_fora)

        # Probabilidade desse placar exato (eventos independentes = multiplica)
        p_placar = p_casa * p_fora

        # Classifica o resultado
        if gols_casa > gols_fora:
            p_vitoria_casa += p_placar
        elif gols_casa == gols_fora:
```

```
        p_empate      += p_placar
    else:
        p_vitoria_fora += p_placar

# Resultado para Noruega x França ( $\lambda_{noruega}=1.52$ ,  $\lambda_{franca}=2.83$ ):
# Noruega: 18.6% | Empate: 16.6% | França: 64.8%
```

16. Mergulho no Código — Parte 6: Monte Carlo Detalhado

16.1 Por que 50.000 simulações?

Quanto mais simulações, mais precisa é a estimativa. Com 50.000, o erro estatístico é inferior a 0,5 ponto percentual — suficiente para o nosso propósito. Rodar 1 milhão seria mais preciso, mas demoraria 20x mais.

```
import random

from collections import defaultdict

N_SIMULACOES = 50_000

campeoes = defaultdict(int)  # conta vitórias de cada time

for simulacao in range(N_SIMULACOES):

    # ■■ 1. Cópia da tabela atual (para não alterar o original) ■■
    tabela = copiar_tabela_atual()

    # ■■ 2. Simula jogos pendentes da fase de grupos ■■
    for jogo in jogos_pendentes:
        gc, gf = amostrar_poisson(lambda_casa), amostrar_poisson(lambda_fora)
        atualizar_tabela(tabela, jogo, gc, gf)

    # ■■ 3. Classifica os 12 grupos ■■
    classificados = []
    for grupo in tabela:
        ranking = sorted(grupo, key=lambda t: (-pontos[t], -saldo[t]))
        classificados.append(ranking[:2])  # top 2 de cada grupo

    # ■■ 4. Pega os 8 melhores 3os colocados ■■
    terceiros = sorted(todos_terceiros, key=lambda t: -pontos[t])[:8]
    fase32 = todos_primeiros + todos_segundos + terceiros

    # ■■ 5. Simula o mata-mata ■■
    rodada = fase32
    while len(rodada) > 1:
        proxima = []
        for i in range(0, len(rodada), 2):
            vencedor = simular_eliminatorio(rodada[i], rodada[i+1])
            proxima.append(vencedor)
        rodada = proxima

    # ■■ 6. Registra o campeão ■■
    campeoes[rodada[0]] += 1

# Calcula probabilidades finais
for time, vitorias in sorted(campeoes.items(), key=lambda x: -x[1]):
    print(f"{time}: {vitorias/N_SIMULACOES*100:.1f}%")
```

16.2 Função de simulação eliminatória

```
def simular_eliminatorio(time_a, time_b):  
    gc, gf = amostrar_poisson(lambda_casa), amostrar_poisson(lambda_fora)  
  
    if gc > gf:  
        return time_a # time_a venceu no tempo normal  
    if gf > gc:  
        return time_b # time_b venceu no tempo normal  
  
    # Empate → pênaltis (proporcional à força ofensiva)  
    forca_a = ataque.get(time_a, 1.0)  
    forca_b = ataque.get(time_b, 1.0)  
    proba_a = forca_a / (forca_a + forca_b)  
  
    return time_a if random.random() < proba_a else time_b
```


17. Mergulho no Código — Parte 7: Boas Práticas Usadas

17.1 Tratamento de valores ausentes

A API às vezes retorna **None** (nulo) em vez de um número. Sem tratamento, operações matemáticas quebrariam. Usamos o padrão *or 0*:

```
# Sem tratamento – risco de erro:
gols = jogo["Home"]["Score"] # pode ser None
total = gols + 1             # TypeError: NoneType + int

# Com tratamento seguro:
def num(x):
    try:
        return int(x)
    except (TypeError, ValueError):
        return None # devolve None em vez de quebrar

gols = num(jogo["Home"].get("Score")) or 0
# num() converte para int; "or 0" transforma None em 0
```

17.2 Pausa entre requisições (rate limiting)

Fazer 64 requisições seguidas sem pausa pode sobrecarregar o servidor e fazer com que ele bloqueie nosso acesso. Por isso, pausamos 0.25 segundos entre cada requisição:

```
import time

for jogo in jogos:
    dados = sessao.get(url_do_jogo)
    processar(dados)
    time.sleep(0.25) # pausa de 250ms entre cada requisição
# 64 jogos × 0.25s = ~16 segundos no total (muito razoável)
```

17.3 Usando os arquivos na mesma pasta do script

```
import os

# __file__ = caminho completo deste arquivo Python
# dirname() = pega só a pasta onde ele está
pasta = os.path.dirname(os.path.abspath(__file__))

# Monta caminhos completos para os arquivos de saída
arquivo_json = os.path.join(pasta, "fifa_jogos_completo.json")
arquivo_csv = os.path.join(pasta, "fifa_jogos.csv")

# Assim, os arquivos são sempre salvos na mesma pasta do script,
# independente de onde o script é executado no terminal
```

17.4 f-strings — texto dinâmico em Python

As f-strings (strings com f antes das aspas) permitem inserir variáveis diretamente no texto:

```
nome = "Messi"
gols = 5
pais = "Argentina"

# Forma antiga (concatenação):
mensagem = nome + " (" + pais + ") marcou " + str(gols) + " gols"

# Com f-string (muito mais legível):
mensagem = f"{nome} ({pais}) marcou {gols} gols"
# → "Messi (Argentina) marcou 5 gols"

# Também permite formatação:
porcentagem = 0.648
print(f"França: {porcentagem:.1%}") # → "França: 64.8%"
print(f"Gols: {gols:>5}") # → "Gols: 5" (alinhado à direita)
```

18. Visão Geral Final — Tudo Conectado

Agora que você entendeu cada parte, veja como tudo se encaixa. O projeto tem três camadas principais:

Camada	O que faz	Tecnologia	Arquivos gerados
Coleta	Busca dados brutos na API da FIFA	requests + JSON	fifa_jogos_completo.json
Processamento	Limpa, filtra e transforma os dados	Python puro + csv + regex	fifa_jogos.csv fifa_artilharia.csv fifa_destaque_jogo.csv
Análise	Calcula probabilidades, simula torneio e ranking	Poisson + Monte Carlo	fifa_palpite_hoje.csv fifa_previsao_campeao_2026.csv

Sequência recomendada para rodar os scripts

Ordem	Script	O que faz	Pré-requisito
1º	fifa_extrair_jogos.py	Baixa todos os jogos e salva JSON/CSV	Nenhum
2º	fifa_jogos_hoje.py	Gera palpites e CSV dos jogos do dia	JSON do passo 1
3º	fifa.ipynb	Exibe gráficos históricos no Jupyter Notebook	JSON do passo 1

Principais bibliotecas Python utilizadas

Biblioteca	Para que serve	Como instalar
requests	Fazer requisições HTTP (buscar dados da API)	pip install requests
json	Ler e salvar arquivos JSON	já vem no Python
csv	Ler e salvar planilhas CSV	já vem no Python
math	Cálculos matemáticos (fatorial, exponencial)	já vem no Python
datetime	Manipulação de datas e horários	já vem no Python
collections	defaultdict para acumuladores sem erro	já vem no Python
re	Expressões regulares para extrair texto	já vem no Python
random	Geração de números aleatórios (Monte Carlo)	já vem no Python
reportlab	Geração de arquivos PDF	pip install reportlab